

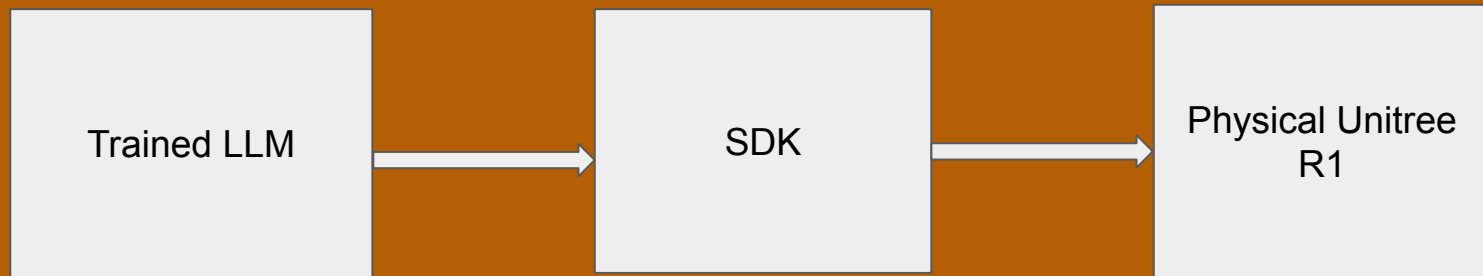
Unitree Week 14+15

Presentation by Mario Ortiz

This Week

- Researched how to deploy trained models onto the Unitree R1 robot.

Primary Steps



Requirements

- Unitree R1 EDU Version
 - Access secondary development tools, the onboard Jetson Orin computer.
- Software Development Kit Setup (Unitree SDK2 or Unitree_rl_gym): Bridge between the trained model and the Unitree R1.
- Environment (SSH): allows for remote access, file transfer, environment management for sim-to-real.
- The converted model: The trained model (generally trained with PyTorch or TensorFlow) must be converted to an ONNX or TensorRT file.
 - These specific formats let you run the model in the onboard GPU.

ONNX vs. TensorRT

- | | |
|--|--|
| <ul style="list-style-type: none">● ONNX (Open Neural Network Exchange):<ul style="list-style-type: none">○ Considered an “open”, intermediate format for model portability across different deep learning frameworks and hardware.○ Not as optimized as TensorRT on NVIDIA hardware. | <ul style="list-style-type: none">● TensorRT<ul style="list-style-type: none">○ High performance, proprietary SDK for optimizing and deploying deep learning models.○ Exclusively for NVIDIA GPU (required CUDA). |
|--|--|

Converting trained model to an ONNX file

- Trained models from Reinforcement Learning on Isaac Labs are PyTorch files.
- Online resources provide methods to convert PyTorch files to both an ONNX file and TensorRT file.
- Due to the NVIDIA GPU on the Unitree R1, we will use TensorRT.

model_0.pt	2/12/2026 1:10 PM	PT File	7,659 KB
model_50.pt	2/12/2026 1:12 PM	PT File	7,660 KB
model_100.pt	2/12/2026 1:14 PM	PT File	7,660 KB
model_150.pt	2/12/2026 1:16 PM	PT File	7,660 KB
model_200.pt	2/12/2026 1:18 PM	PT File	7,660 KB
model_250.pt	2/12/2026 1:20 PM	PT File	7,660 KB
model_300.pt	2/12/2026 1:22 PM	PT File	7,660 KB
model_350.pt	2/12/2026 1:24 PM	PT File	7,660 KB
model_400.pt	2/12/2026 1:26 PM	PT File	7,660 KB
model_450.pt	2/12/2026 1:28 PM	PT File	7,660 KB
model_500.pt	2/12/2026 1:30 PM	PT File	7,660 KB
model_550.pt	2/12/2026 1:31 PM	PT File	7,660 KB

```
py Copy
import torch.onnx

#Function to Convert to ONNX
def Convert_ONNX():

    # set the model to inference mode
    model.eval()

    # Let's create a dummy input tensor
    dummy_input = torch.randn(1, input_size, requires_grad=True)

    # Export the model
    torch.onnx.export(model,          # model being run
                      dummy_input,   # model input (or a tuple for multiple inputs)
                      "ImageClassifier.onnx",  # where to save the model
                      export_params=True,  # store the trained parameter weights inside
                                           # the ONNX version to export the model to
                      do_constant_folding=True,  # whether to execute constant folding
                      input_names = ['modelInput'],  # the model's input names
                      output_names = ['modelOutput'],  # the model's output names
                      dynamic_axes={'modelInput': {0: 'batch_size'},  # variable length
                                     'modelOutput': {0: 'batch_size'}})

    print(" ")
    print('Model has been converted to ONNX')
```

Setup Information

- Transfer TensorRT file into R1 using SCP or SFTP.
- An inference script (bridges the RL policy/Imitation Learning model with the robot's hardware interface) must also be transferred into the R1.
- Implement SDK2 to the inference script → allows to read sensor data and send commands to the motors.
- Control Loop Development:
 - Subscribe to robot states
 - Process sensor data
 - Run inferences through the model
 - Send motion commands to the R1 with Data Distribution Service.
- Once all this is done, we can run the script on the R1's computer.

Making an inference script

- The original Python code used to design the model is first implemented into the script.
- Model weights are then loaded (instantiated and saved from the TensorRT model).
- Evaluation code is implemented.
- Input data is implemented.
- Running inference commands to get predictions.

This is similar code structure as when we tested Fine Tuned Deepseek Models for the SMLM project.

Setting Up SDK2

- Ensure the prebuild environment (Operating System; Ubuntu, CPU; aarch64 and x86_64; Compiler; gcc version 9.4.0)
- Environment Setup: requires these dependencies: CMake version 3.10 or higher, gcc version 9.4.0, Make

Build examples

To build the examples inside this repository:

```
mkdir build
cd build
cmake ..
make
```

Installation

To build your own application with the SDK, you can install the unitree_sdk2 to your system directory:

```
mkdir build
cd build
cmake ..
sudo make install
```

Or install unitree_sdk2 to a specified directory:

```
mkdir build
cd build
cmake .. -DCMAKE_INSTALL_PREFIX=/opt/unitree_robotics
sudo make install
```

unitree_sdk2 / example / r1 /

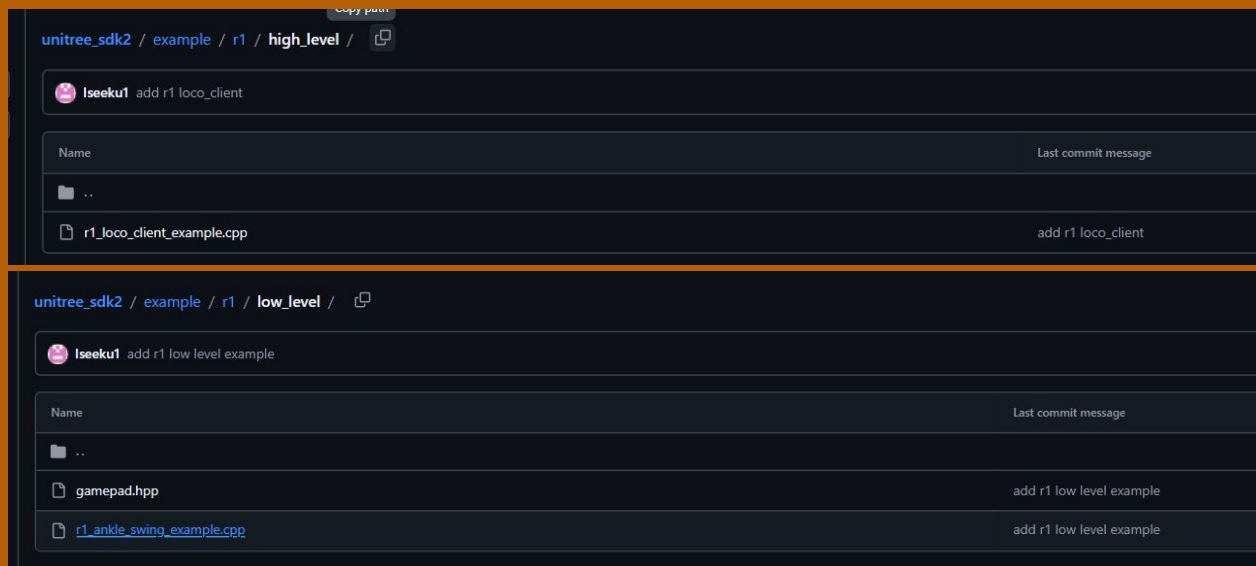
 **iseekur1** add r1 low level example

Name	Last commit message
..	
high_level	add r1 loco_client
low_level	add r1 low level example
CMakeLists.txt	add r1 low level example

SDK2 already has some low level
Unitree R1 examples.

Setting Up SDK2

- There is some SDK2 examples using Unitree R1.
- The Low Level example “r1_ankle_swing_example.cpp” bypasses high level control, talking directly to Motor SDK
- In this example Motor SDK oscillates the ankle joint by sending raw position and gain targets (K_p , K_d).
- The high level “r1_loco_client_example.cpp” sends commands such as Move(), Stop(), StandUp().



r1_loco_client_example.cpp

High Level Locomotion commands such as walking, standing, or state checking (FSMs) to the robot are possible with this program.

The program takes terminal inputs and passes them to the robot.

ChannelFactory and LocoClient are from the Unitree SDK2 communication infrastructure.

Commands in this file for joint movement: get_fsm_id, mode, set_velocity, stand_up, move, damp, dero_torque, switch_move_mode

unitree_sdk2 / example / r1 / high_level / r1_loco_client_example.cpp

lokeul add r1_loco_client

```
Code Blame 189 lines (144 loc) · 4.39 KB

1 #include <chrono>
2 #include <cstring>
3 #include <thread>
4
5 #include "unitree/robot/r1/loco/r1_loco_api.h"
6 #include "unitree/robot/r1/loco/r1_loco_client.h"
7
8 std::vector<float> stringToFloatVector(const std::string& str) {
9     std::vector<float> result;
10     std::stringstream ss(str);
11     float num;
12     while (ss >> num) {
13         result.push_back(num);
14         // ignore any trailing whitespace
15         ss.ignore();
16     }
17     return result;
18 }
19
20 int main(int argc, char const* argv[]) {
21     std::map<std::string, std::string> args = {{"network_interface", "lo"}};
22
23     std::map<std::string, std::string> values;
24     for (int i = 1; i < argc; ++i) {
25         std::string arg = argv[i];
26         if (arg.substr(0, 2) == "--") {
27             size_t pos = arg.find("=");
28             std::string key, value;
29             if (pos != std::string::npos) {
30                 key = arg.substr(2, pos - 2);
31                 value = arg.substr(pos + 3);
32             }
33             if (value.front() == "'" && value.back() == "'") {
34                 value = value.substr(1, value.length() - 2);
35             }
36         } else {
37             key = arg.substr(2);
38             value = "";
39         }
40         if (args.find(key) != args.end()) {
41             args[key] = value;
42         } else {
43             args.insert({key, value});
44         }
45     }
46
47     std::string network_interface = args["network_interface"];
48     unitree::robot::ChannelFactory::Instance()->Init(0, network_interface);
49
50     unitree::robot::r1::LocoClient client;
```

```
51
52 client.Init();
53 client.SetTimeout(10, 0);
54
55 for (const auto& arg_pair : args) {
56     std::cout << "Processing command: [" << arg_pair.first << "] with param: [" << arg_pair.second << "] ...";
57     << std::endl;
58     if (arg_pair.first == "network_interface") {
59         continue;
60     }
61
62     if (arg_pair.first == "get_fsm_id") {
63         int fsm_id;
64         client.GetFsmId(fsm_id);
65         std::cout << "Current fsm_id: " << fsm_id << std::endl;
66     }
67
68     if (arg_pair.first == "get_fsm_mode") {
69         int fsm_mode;
70         client.GetFsmMode(fsm_mode);
71         std::cout << "Current fsm_mode: " << fsm_mode << std::endl;
72     }
73
74     if (arg_pair.first == "set_fsm_id") {
75         int fsm_id = std::stoi(arg_pair.second);
76         client.SetFsmId(fsm_id);
77         std::cout << "set_fsm_id to " << fsm_id << std::endl;
78     }
79
80     if (arg_pair.first == "set_velocity") {
81         std::vector<float> param = stringToFloatVector(arg_pair.second);
82         auto param_size = param.size();
83         float vx, vy, omega, duration;
84         if (param_size == 3) {
85             vx = param.at(0);
86             vy = param.at(1);
87             omega = param.at(2);
88             duration = 1.f;
89         } else if (param_size == 4) {
90             vx = param.at(0);
91             vy = param.at(1);
92             omega = param.at(2);
93             duration = param.at(3);
94         } else {
95             return << "Invalid param size for method SetVelocity: " << param_size << std::endl;
96         }
97         return;
98     }
99     client.SetVelocity(vx, vy, omega, duration);
100     std::cout << "set velocity to " << arg_pair.second << std::endl;
101 }
102
103 if (arg_pair.first == "damp") {
104     client.Dump();
105 }
106
107 if (arg_pair.first == "start") {
108     client.Start();
109 }
110
111 // If (arg_pair.first == "equal") {
112 //     client.Equal();
113 // }
114
115 // client.Equal();
116 // }
117
118 // If (arg_pair.first == "sit") {
119 //     client.Sit();
120 // }
121
122 if (arg_pair.first == "stand_up") {
123     client.StandUp();
124 }
125
126 if (arg_pair.first == "zero_torque") {
127     client.ZeroTorque();
128 }
129
130 if (arg_pair.first == "stop_move") {
131     client.StopMove();
132 }
133
134 if (arg_pair.first == "switch_move_mode") {
135     bool flag;
136     if (arg_pair.second == "true") {
137         flag = true;
138     } else if (arg_pair.second == "false") {
139         flag = false;
140     } else {
141         return << "Invalid argument: " << arg_pair.second << std::endl;
142     }
143     client.SwitchMoveMode(flag);
144 }
145
146 if (arg_pair.first == "move") {
147     std::vector<float> param = stringToFloatVector(arg_pair.second);
148     auto param_size = param.size();
149     float vx, vy, omega;
150     if (param_size == 3) {
151         vx = param.at(0);
152         vy = param.at(1);
153         omega = param.at(2);
154     } else {
155         return << "Invalid param size for method SetVelocity: " << param_size << std::endl;
156     }
157     return;
158 }
159
160 if (arg_pair.first == "set_speed_mode") {
161     int param = std::stoi(arg_pair.second);
162     client.SetSpeedMode(param);
163     std::cout << "set speed mode to " << arg_pair.second << std::endl;
164 }
165
166 std::cout << "Done!" << std::endl;
167 }
168
169 return 0;
170 }
```

Using r1_loco_client_example.cpp

This code can be used to do our specific tasks. For example, if we want the Unitree R1 to walk a specific number of steps, we could use this code, sending a sequence of commands to make the robot do specific tasks with the joints.

`client.StandUp()` → robot stabilizes itself to stand

`client.SetVelocity(0.1, 0, 0, 1.0)` → Walk forward at 0.1m/s for 1 second.

`client.StopMove()` → stops walking

r1_ankle_swing_example.cpp

Unlike the other high level code (which tells the R1 to do higher level tasks like walking) this program is more detailed, controlling all 26 dof's of the R1.

- The example uses mathematical control modes to run a sequence of actions. Directly uses direct motor mathematics to do the task.
- As this code is more detailed, learning how to utilize this code allows us to make the R1 do very specific tasks.
 - However, developing our own design based on the low level code is more dangerous than high level as the Robot will fail if the math computations are incorrect (while with the High level code, we can simply tweak the commands).

High Level vs. Low Level Code

The High Level code is better to use if we want to use it's specific tasks such as walking.

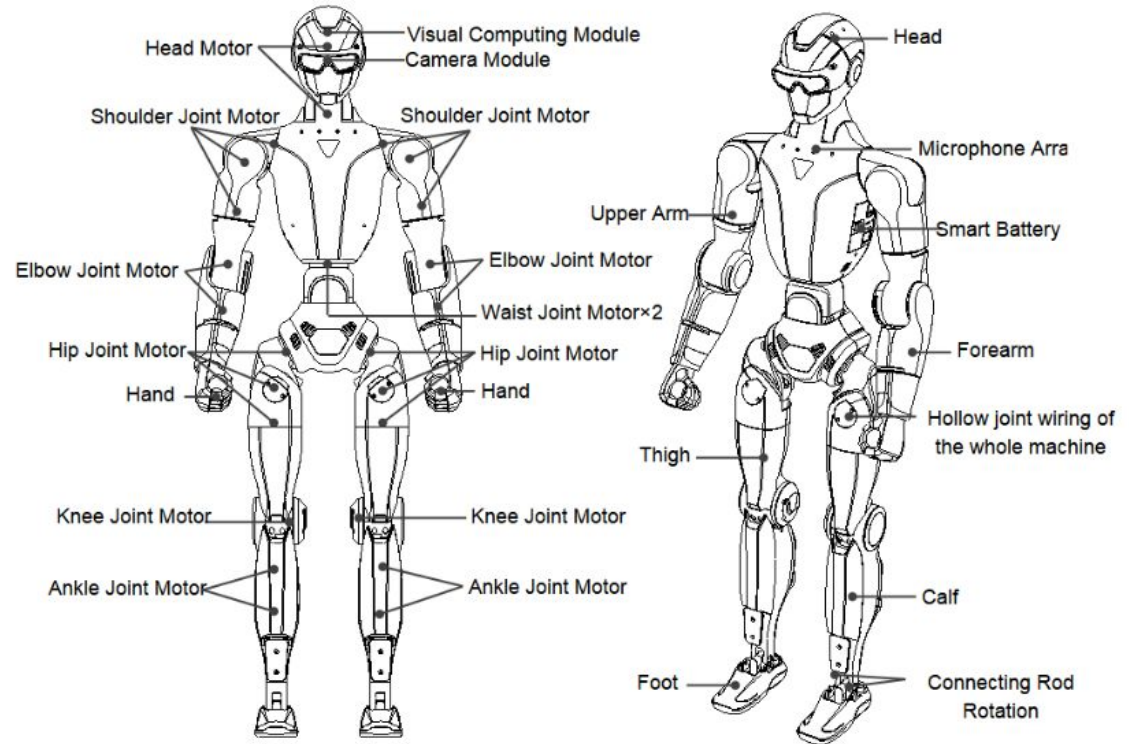
- Problem with High Level code is it is less customizable. You basically can only use the modes they have provided for you.

The Low Level code is better to use if we want to make the R1 do specific but simple tasks (especially related to hand gestures).

- Of course, with the logic of this code, we can attempt to do more complicated tasks with it.
- To do these tasks, we need to calculate the trigonometry for each joint required for the task, ensure mathematically that the center of mass of the robot is maintained, and ensure to sync the motors of each joint simultaneously.

Electrical Interface of the R1

Type	R1 Air	R1 Basic & R1-EDU
Total DOF	20	26
Single Leg DOF	6	6
Waist DOF	0	2
Single Arm DOF	4	5
Head DOF	0	2



Electrical Interface of the R1

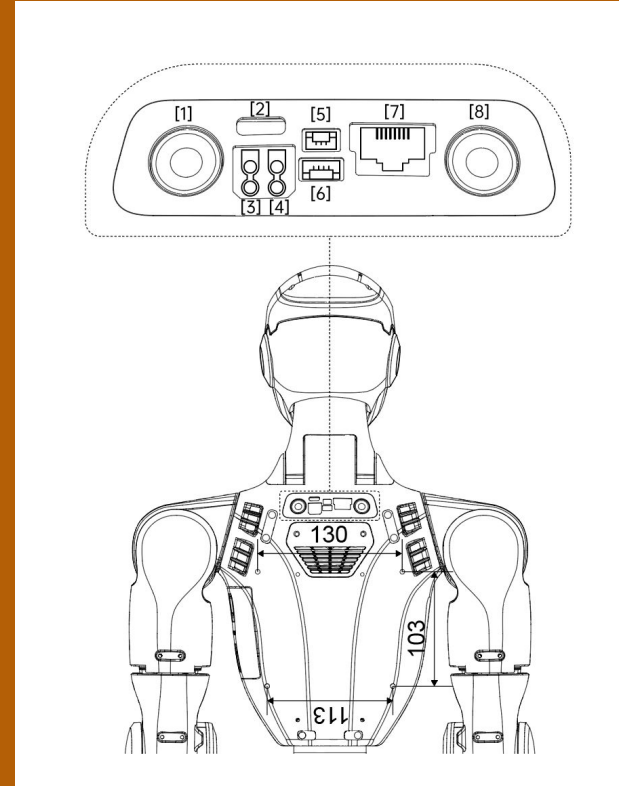
- Dexterous Hand → if learn to use this can train the R1 to do very specific tasks with its hands.

Dexterous Hand Dex3-1

Dexterous Hand Dex3-1 Electrical Parameters	
Rendering	
Working Voltage	12-58V
Perception Range	10g-2500g
Degrees of Freedom (DOF)	Total DOF: 7 1. Thumb: 3 active DOFs; 2. Index finger: 2 active DOFs; 3. Middle finger: 2 active DOFs.
Joint Angles	Thumb: 0°~+100°, -35°~+60°, -60°~+60°; Index, Middle finger: 0°~+90°, 0°~+100°
Array Sensor Quantity	9

Electrical Interface of the R1

- Connects body joint motors, sensor peripherals, network ports.
- Allows convenient debugging, troubleshooting, and secondary development



Electrical Interface Description

- Button to stand up the R1
- Provides information on the required power and other connections the R1 requires

N o.	Interface T ype	Abbreviation	Interface Description
1	Button	Stand up / Lie down	After a short press, the action will be executed 3 seconds after the voice announcement is completed.
2	Type-C	Type-C	Supports USB 3.0 devices Used in EDU version for connecting rear computing modules
3	XT30UPB-F	24V	24V/3A Power Output
4	XT30UPB-F	36V	36V/5A Power Output
5	GH1.25	E-Stop Interface	External Emergency Stop Interface See below for pin definitions from left to right
6	GH1.25	Reserved Socket	NC
7	RJ45	1000 BASE-T	Gigabit Ethernet
8	Reserved Button	Reserved Button	NC

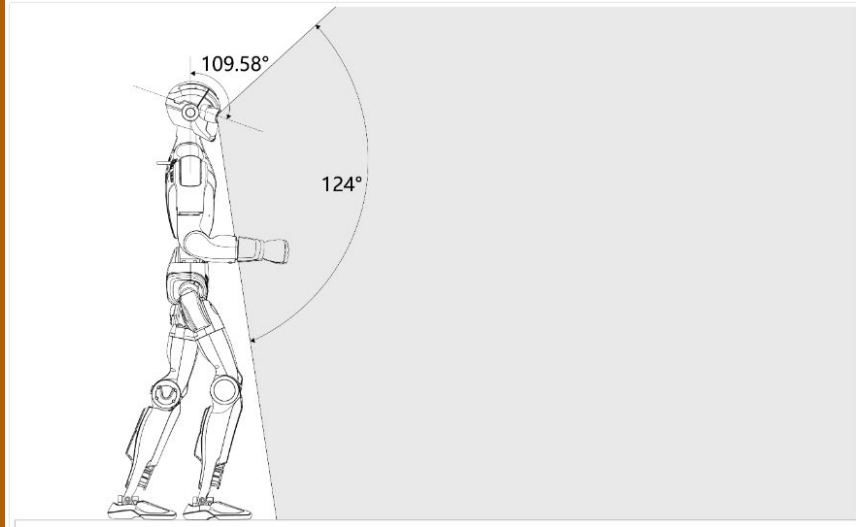
Description on R1-EDU's onboard computer

Onboard Computer

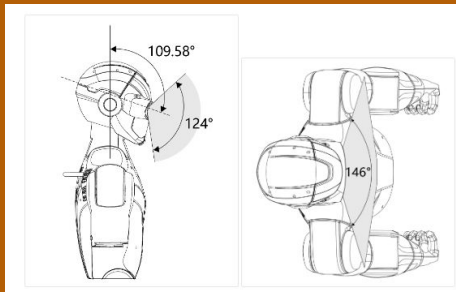
The R1-EDU Professional version comes standard with one [Development Computing Unit].

Parameter	Development Computing Unit (PC)
Model	Jetson Orin NX
CPU	Arm® Cortex®-A78AE
Cores	8
Threads	8
Max Turbo Frequency	2GHz
Video Memory	16G
RAM	16G
Cache	2MB L2 + 4MB L3
Storage	2T
Intel® Image Processing Unit	6.0
GPU	1024-core NVIDIA Ampere architecture GPU with 32 Tensor Cores
Max Dynamic Frequency	918MHz
GNA	3.0
Intel® DL Boost	Yes
Intel® Adaptix™ Technology	Yes
Intel® Hyper-Threading	Yes
Instruction Set	64bit
OpenGL	4.6
OpenCL	3.0
DirectX	12.1
IP Address	192.168.123.164

Description on R1-EDU's camers



Category	Parameter
RGB Full Resolution	1280*1088
Depth Resolution	544*448
Depth Map Frame Rate	10Hz
RGB Frame Rate	30Hz
Resolution	13 Million Pixels
Features	Global Exposure High Sensitivity Supports High Dynamic Range Output High SNR 850nm/940nm NIR Enhancement Supports External Exposure Control and Multi-sensor Sync Auto Exposure/Gain Control Supports LED STROBE H/V Window Adjustment 15.5x Analog Gain, 16x Digital Gain I2C Interface Programming
Dynamic Range	Normal Mode 66dB HDR Mode >100dB



Join Motor Names + Limitations

Joint Sequence Names and Limits

Joint No.	Joint Name	Limit (Radians)
0	L_LEG_HIP_PITCH	-2.9322~2.5482
1	L_LEG_HIP_ROLL	-1.0472~1.7453
2	L_LEG_HIP_YAW	-2.7402~2.7402
3	L_LEG_KNEE	-0.1745~2.4260
4	L_LEG_ANKLE_PITCH	-0.8727~0.5760
5	L_LEG_ANKLE_ROLL	-0.2618~0.2618
6	R_LEG_HIP_PITCH	-2.9322~2.5482
7	R_LEG_HIP_ROLL	-1.0472~1.7453
8	R_LEG_HIP_YAW	-2.7402~2.7402
9	R_LEG_KNEE	-0.1745~2.4260
10	R_LEG_ANKLE_PITCH	-0.8727~0.5760
11	R_LEG_ANKLE_ROLL	-0.2618~0.2618
12	WAIST_YAW	-2.618~2.618
13	WAIST_ROLL	-0.5236~0.5236
14	EMPTY	EMPTY
15	L_SHOULDER_PITCH	-3.1416~2.0944

16	L_SHOULDER_ROLL	-2.4784~0.2269
17	L_SHOULDER_YAW	-1.9199~1.9199
18	L_ELBOW	-0.9757~2.1850
19	L_WRIST_ROLL	-1.9199~1.9199
20	EMPTY	EMPTY
21	EMPTY	EMPTY
22	R_SHOULDER_PITCH	-3.1416~2.0944
23	R_SHOULDER_ROLL	-2.4784~0.2269
24	R_SHOULDER_YAW	-1.9199~1.9199
25	R_ELBOW	-0.9757~2.1850
26	R_WRIST_ROLL	-1.9199~1.9199
27	EMPTY	EMPTY
28	EMPTY	EMPTY
29	HEAD_PITCH	-0.6283~0.6283
30	HEAD_YAW	-1.8326~1.8326

Other important Robot Specifications

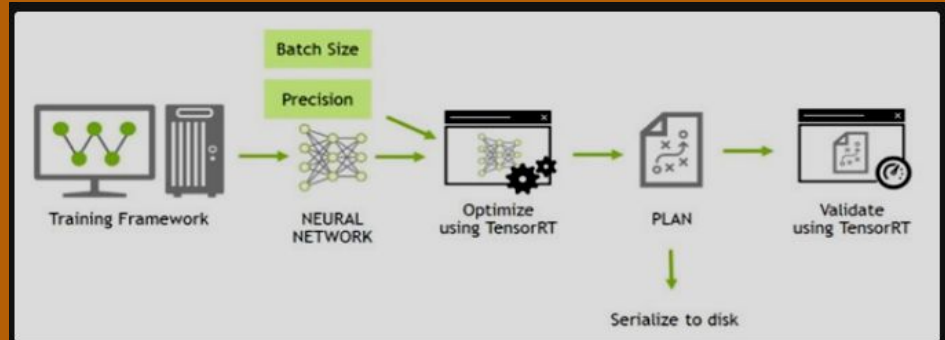
Robot Specifications

Model	R1	R1 Basic	R1-EDU
Dimensions (Standing)	1230x357x190mm	1230x357x190mm	1230x357x190mm
Dimensions (Folded)	690x450x300mm	690x450x300mm	690x450x300mm
Weight (with battery)	Approx. 27kg	Approx. 29kg	Approx. 29kg
Total DOF	20	26	26
Single Leg DOF	6	6	6
Waist DOF	0	2	2
Single Arm DOF	4	5	5
Dexterous Hand	/	/	Optional
Joint Output Bearing	Cross Roller + Double Row Ball	Cross Roller + Double Row Ball	Cross Roller + Double Row Ball
Joint Motor	Low Inertia High Speed Inner Motor PMSM (Better response and cooling)	Low Inertia High Speed Inner Motor PMSM (Better response and cooling)	Low Inertia High Speed Inner Motor PMSM (Better response and cooling)
Max Arm Payload [1]	Approx. 2kg	Approx. 2kg	Approx. 2kg
Lower + Upper Leg Length	0.6M	0.6M	0.6M
Arm Span	0.435M	0.435M	0.435M
Large Joint Range	Knee: -10°~+139° Hip: Y:±157°, P:-168°~+146°, R:-60°~+100°	Waist: Y:±150°, R:±30° Knee: -10°~+139° Hip: Y:±157°, P:-168°~+146°, R:-60°~+110°	Waist: Y:±150°, R:±30° Knee: -10°~+139° Hip: Y:±157°, P:-168°~+146°, R:-60°~+110°
Wiring Method	Hollow Shaft + Internal Wiring	Hollow Shaft + Internal Wiring	Hollow Shaft + Internal Wiring
Joint Encoder	Dual Encoders	Dual + Single Encoder	Dual + Single Encoder
Cooling System	Localized Air Cooling	Localized Air Cooling	Localized Air Cooling
Power Supply	Lithium Battery	Lithium Battery	Lithium Battery

Basic Computing	8-core High Performance CPU	8-core High Performance CPU	8-core High Performance CPU
Mic Array	4-array	4-array	4-array
Speaker	Yes	Yes	Yes
WiFi 6, Bluetooth 5.2	Yes	Yes	Yes
Camera Module	Monocular Camera	Binocular Camera	Binocular Camera
High Computing Module	/	/	Optional NVIDIA Jetson Orin (40-10 TOPS)
Smart Battery (Quick Release)	Yes	Yes	Yes
Charger	Yes	Yes	Yes
Handheld Remote	Yes	Yes	Yes
Battery Life	Approx. 1h	Approx. 1h	Approx. 1h
Smart OTA Upgrade	Yes	Yes	Yes
Secondary Development [2]	/	/	Yes
Warranty Period [3]	6 Months	8 Months	12 Months

TensorRT Documentation

- Provides more information on the importance of TensorRT for projects such as this and how to convert neural network files (such as PyTorch) to this file.



Unitree_RL_gym

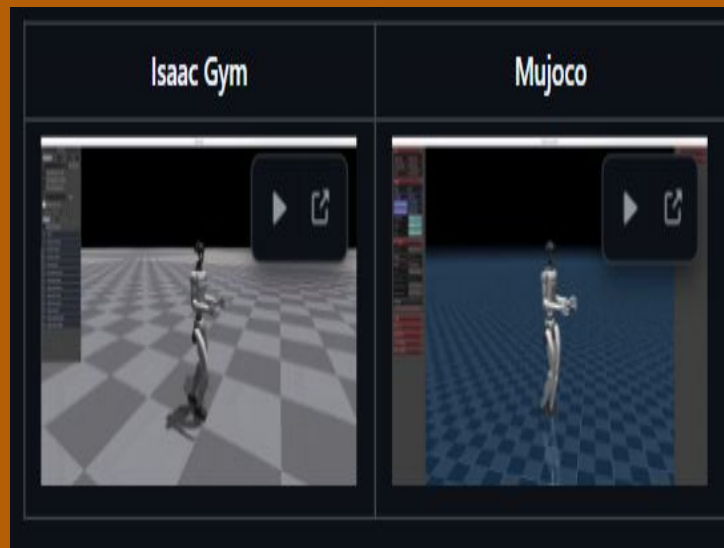
- We have already discussed implementation of SDK2 as the bridge between the trained R1 model's with the actual hardware.
- Unitree_RL_gym is the other option, and may be the better option during the beginning stages of deployment, as it uses already made software to deploy the model, while SDK2 must be built from the ground up.
 - SDK2 is better for more advanced integration and for better control of the control of the robot.
- Also, Unitree_RL_gym is easily integratable with Isaac Sim + Isaac Lab
- *Therefore, I will most likely start writing deployment code/setting up the deployment with Unitree_RL_gym, and then later on set it up with SDK2 when we are working on more advanced tasks.*

Unitree_RL_gym Github Repo

- “Repository for reinforcement learning implementation based on Unitree robots, supporting Unitree Go2, H1, H1_2, and G1”.
- Process Overview: Train → Play → Sim2Sim → Sim2Real
 - Train using its Gym simulation env
 - Play command: testing the final trained policy
 - Sim2Sim: Testing the final trained policy to other simulators (ensures it won't only work on one simulator)
 - Sim2Real: deploying the policy to physical robot.
- *Concept so far seems integratable with IsaacSim, though they use Isaac Gym*

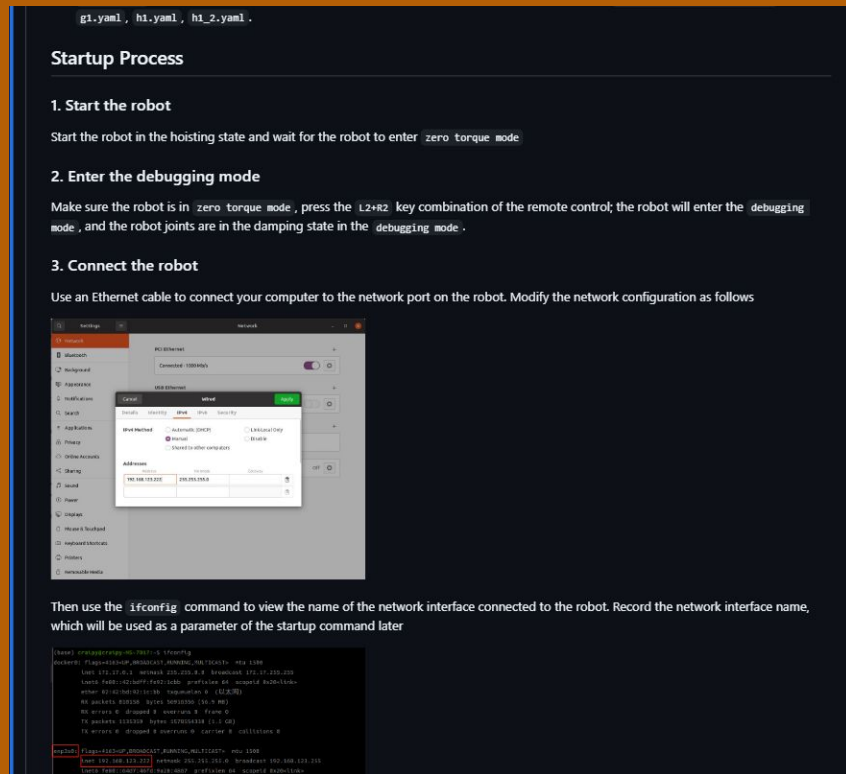
Sim2Sim

- In this repository, they utilize Isaac Gym to train and play the model, and then use MuJoCo as there secondary simulator for Sim2Sim protocol.
- As we already have RL code using IsaacSim, the same process can be done by transferring IsaacSim+IsaacLab trained RL models onto MuJoCo.



Sim2Real (Physical Deployment)

- The repository provides a simple method to deploy the model.
- Steps involve starting the robot, put robot into *debugging mode*, connecting the robot, and then starting the program.



Sim2Real (Physical Deployment)

- For “4. Start the program”, they discuss a *network card*; also they utilize a .yaml file which is a controller file for the robot.
- Once program is started, they discuss putting the robot into *Zero torque state* and *default position state*.
- Then, you can start *Motion control mode*, which allows you to control the robot’s movements.

4. Start the program

Assume that the network card currently connected to the physical robot is named `enp3s0`. Take the G1 robot as an example, execute the following command to start

```
python deploy_real.py enp3s0 g1.yaml
```

4.1 Zero torque state

After starting, the robot joints will be in the zero torque state. You can shake the robot joints by hand to feel and confirm.

4.2 Default position state

In the zero torque state, press the `start` button on the remote control, and the robot will move to the default joint position state.

After the robot moves to the default joint position, you can slowly lower the hoisting mechanism to let the robot's feet touch the ground.

4.3 Motion control mode

After the preparation is completed, press the `A` button on the remote control, and the robot will step on the spot. After the robot is stable, you can gradually lower the hoisting rope to give the robot a certain amount of space to move.

At this time, you can use the joystick on the remote control to control the movement of the robot. The front and back of the left joystick controls the movement speed of the robot in the x direction The left and right of the left joystick controls the movement speed of the robot in the y direction The left and right of the right joystick controls the movement speed of the robot's yaw angle

4.4 Exit control

In motion control mode, press the `select` button on the remote control, the robot will enter the damping mode and fall down, and the program will exit. Or use `ctrl+c` in the terminal to close the program.

Note:

Since this example deployment is not a stable control program and is only used for demonstration purposes, please try not to disturb the robot during the control process. If any unexpected situation occurs during the control process, please exit the control in time to avoid danger.

Transferring Robot Learning Policies from Simulation to Reality

- This documentation is a more in-depth course that teaches how to deploy trained policies to physical robots.

4. Start the program

Assume that the network card currently connected to the physical robot is named `enp3s0`. Take the G1 robot as an example, execute the following command to start

```
python deploy_real.py enp3s0 g1.yaml
```

4.1 Zero torque state

After starting, the robot joints will be in the zero torque state. You can shake the robot joints by hand to feel and confirm.

4.2 Default position state

In the zero torque state, press the `start` button on the remote control, and the robot will move to the default joint position state.

After the robot moves to the default joint position, you can slowly lower the hoisting mechanism to let the robot's feet touch the ground.

4.3 Motion control mode

After the preparation is completed, press the `A` button on the remote control, and the robot will step on the spot. After the robot is stable, you can gradually lower the hoisting rope to give the robot a certain amount of space to move.

At this time, you can use the joystick on the remote control to control the movement of the robot. The front and back of the left joystick controls the movement speed of the robot in the x direction The left and right of the left joystick controls the movement speed of the robot in the y direction The left and right of the right joystick controls the movement speed of the robot's yaw angle

4.4 Exit control

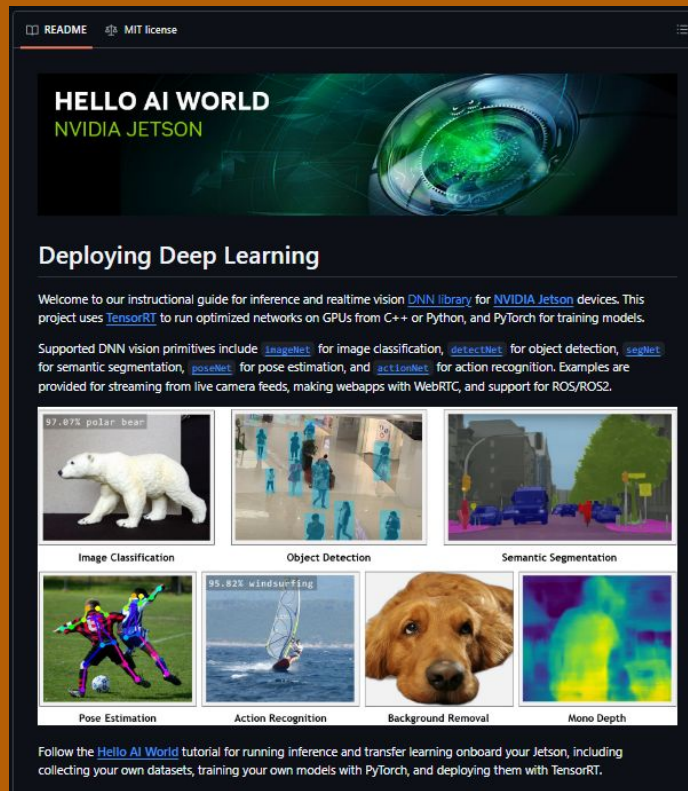
In motion control mode, press the `select` button on the remote control, the robot will enter the damping mode and fall down, and the program will exit. Or use `ctrl+c` in the terminal to close the program.

Note:

Since this example deployment is not a stable control program and is only used for demonstration purposes, please try not to disturb the robot during the control process. If any unexpected situation occurs during the control process, please exit the control in time to avoid danger.

jetson inference

- The jetson inference Github repository provides a tutorial for how to implement inference into NVIDIA Jetson devices (which the Unitree R1 contains).
 - Guide for setting up TensorRT and
 - Guide for other Deep Neural Network Implementations such as imageNet.



unitree_sdk2_python

- This github repository contains more sdk2 examples (for the g1 robot, not r1).

unitree_sdk2_python / example / g1 / high_level /

 **xiaoliangstd** add G1ArmActionClient and corresponding examples 892693e · 10 months ago

Name	Last commit message	Last commit date
...		
 g1_arm5_sdk_dds_example.py	update g1 arm_sdk examples	
 g1_arm7_sdk_dds_example.py	update g1 arm_sdk examples	
 g1_arm_action_example.py	add G1ArmActionClient and corresponding examples	10 months ago
 g1_joco_client_example.py	add g1 5squat2StandUp, StandUp2Squat, Lie2StandUp api	

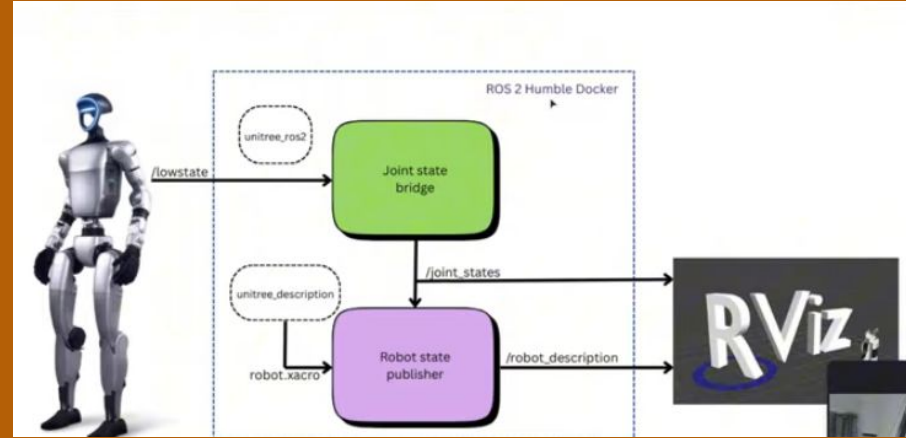
unitree_sdk2_python / example / g1 / low_level /

 **xiaoliangstd** update h1_2 and g1 low_level example 21e44b7 · 2 years ago  History

Name	Last commit message	Last commit date
...		
 g1_low_level_example.py	update h1_2 and g1 low_level example	2 years ago

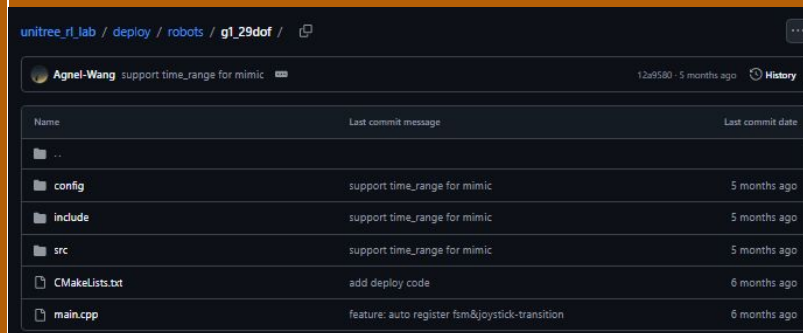
The Construct Robotics Institute

- Provides tutorials for deploying Unitree G1 Robot.
- This image shows the tutorial design for Real-to-Sim → passing the real time movement of the Unitree G1 onto Rviz.



unitree_rl_lab

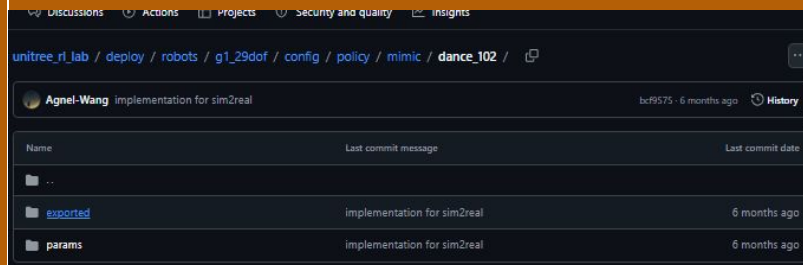
- This Github repository has a “deploy” folder intended for the use of the Unitree G1.
- In `unitree_rl_lab/deploy/robots/g1_29dof`, there is C++ and Python Sim-To-Real scripts.
- The Sim-To-Real script



unitree_rl_lab / deploy / robots / g1_29dof /

Agnel-Wang support time_range for mimic 12a9580 · 5 months ago History

Name	Last commit message	Last commit date
...		
config	support time_range for mimic	5 months ago
include	support time_range for mimic	5 months ago
src	support time_range for mimic	5 months ago
CMakeLists.txt	add deploy code	6 months ago
main.cpp	feature: auto register fsm&joystick-transition	6 months ago



unitree_rl_lab / deploy / robots / g1_29dof / config / policy / mimic / dance_102 /

Agnel-Wang implementation for sim2real bcf9575 · 6 months ago History

Name	Last commit message	Last commit date
..		
exported	implementation for sim2real	6 months ago
params	implementation for sim2real	6 months ago

References

- [https://www.unitree.com/cn/opensource/#:~:text=unitree ros%20is%20a%20ROS%20simulation,unitree sdk2](https://www.unitree.com/cn/opensource/#:~:text=unitree%20ros%20is%20a%20ROS%20simulation,unitree%20sdk2)
- [https://learnopencv.com/how-to-convert-a-model-from-pytorch-to-tensorrt-and-speed-up-inference/#:~:text=First%20of%20all%2C%20let's%20implement,.resnet50\(pretrained=True\)](https://learnopencv.com/how-to-convert-a-model-from-pytorch-to-tensorrt-and-speed-up-inference/#:~:text=First%20of%20all%2C%20let's%20implement,.resnet50(pretrained=True))
- [https://wiki.archlinux.org/title/SCP and SFTP](https://wiki.archlinux.org/title/SCP_and_SFTP)
- [https://github.com/unitreerobotics/unitree rl lab](https://github.com/unitreerobotics/unitree_rl_lab)
- [https://github.com/unitreerobotics/unitree sdk2](https://github.com/unitreerobotics/unitree_sdk2)
-

References

- R1 Specification's
https://support.unitree.com/home/en/R1_developer
- TensorRT Documentation
<https://docs.nvidia.com/deeplearning/tensorrt/latest/getting-started/quick-start-guide.html>
- Unitree_rl_gym github repository:
https://github.com/unitreerobotics/unitree_rl_gym
- Unitree_rl_gym deploy model (Sim2Real):
https://github.com/unitreerobotics/unitree_rl_gym/blob/main/deploy/deploy_real/README.md

References

- Sim2Real with IsaacLab:
<https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-lab/latest/transferring-robot-learning-policies-from-simulation-to-reality/index.html>
-